Java — Interview Questions

Core Java questions asked in fresher technical interviews — platform, strings, exceptions, collections, multithreading and Java 8 — with short answers and small code examples (all compiled and run on Java 21).

Answer like this in the interview

Define in one line → **explain** with one real-world example → add **one detail** (a difference, a use case or a limitation). Stop at 30–45 seconds and let the interviewer ask the follow-up.

Questions marked ★ **MOST ASKED** come up in almost every fresher technical round — learn those first.

Top questions to learn first

JDK vs JRE vs JVM · Why Java is platform independent · == vs equals() · String vs StringBuilder vs StringBuffer · final vs finally vs finalize · Checked vs unchecked exceptions · ArrayList vs LinkedList · How HashMap works · Thread vs Runnable · Java 8 features

1 · Platform basics

JDK vs JRE vs JVM ★ MOST ASKED

Part	What it is	Contains
JVM	Java Virtual Machine — runs bytecode on a specific OS	Class loader, bytecode verifier, interpreter, **JIT compiler**, garbage collector
JRE	Java Runtime Environment — what you need to **run** Java	JVM + core class libraries
JDK	Java Development Kit — what you need to **develop** Java	JRE + compiler (`javac`), debugger, `jar`, `javadoc`

1 **Why is Java platform independent? Is the JVM platform independent?** ★ MOST ASKED

`javac` compiles source code into **bytecode** (`.class`), not machine code. Any OS with a JVM can run that bytecode — **write once, run anywhere**.

The **JVM itself is platform dependent** — there is a different JVM for Windows, Linux and macOS.

2 **Explain** `public static void main(String[] args)`**.**

public — the JVM must call it from outside the class. **static** — called without creating an object. **void** — returns nothing. **main** — the name the JVM looks for. **String[] args** — command-line arguments.

3 **What are Java's key features?**

Object-oriented, platform independent, **robust** (strong type checking, exception handling, garbage collection, no explicit pointers), secure (bytecode verification, no pointer arithmetic), multithreaded, high performance with the JIT compiler.

4 What is the JIT compiler?

The **Just-In-Time** compiler inside the JVM converts frequently executed bytecode (“hot” code) into native machine code at run time, so it runs much faster on later calls.

2 · Data types & strings

5 Primitive types vs wrapper classes? What is autoboxing?

8 primitives: `byte, short, int, long, float, double, char, boolean` — stored as plain values. Each has a **wrapper class** (`Integer`, `Double`, `Character`…) that is an object, can be null, and is needed in collections (`List<Integer>`).

Autoboxing converts a primitive to its wrapper automatically (`Integer x = 5;`); **unboxing** does the reverse.

6 Difference between == and equals()? ★ MOST ASKED

== compares **references** for objects (same memory location) and values for primitives. `equals()` compares **content**, when the class overrides it (String, Integer and collections do).

```
String a = "hi";
String b = new String("hi");
System.out.println(a == b);        // false — different objects
System.out.println(a.equals(b));   // true  — same content
```

7 Why is String immutable? What is the String pool? ★ MOST ASKED

Once created, a String's value cannot change; methods like `concat()` or `toUpperCase()` return a **new** String.

Reasons: **String pool** reuse (literals with the same value share one object), **security** (file paths, URLs, passwords can't be altered), **thread safety** and a cacheable hashCode for HashMap keys.

`String s = "hi"` uses the pool; `new String("hi")` always creates a new object on the heap.

String vs StringBuilder vs StringBuffer ★ MOST ASKED

Point	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized methods)
Speed	Slow for repeated changes	Fastest	Slower than StringBuilder
Use	Fixed text	Building strings in one thread (loops)	Building strings shared across threads

8 Show StringBuilder in use.

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" 21").insert(0, "Hello ");
System.out.println(sb);             // Hello Java 21
System.out.println(sb.reverse());   // 12 avaJ olleH
```

3 · Keywords

final vs finally vs finalize ★ MOST ASKED

Keyword	Type	Meaning
final	Modifier	final variable = constant; final method = cannot be overridden; final class = cannot be extended
finally	Block	Runs after try/catch whether or not an exception occurred — used for cleanup
finalize()	Method	Called by the garbage collector before an object is removed — **deprecated** since Java 9; use try-with-resources instead

9 What do `static`, `this`, `super` and `transient` do?

- **static** — belongs to the class; one shared copy; static blocks run once when the class loads.
- **this** — current object; **super** — parent class members/constructor.
- **transient** — the field is skipped during serialization (e.g. a password).
- **volatile** — reads and writes go to main memory, so every thread sees the latest value.

10 Stack memory vs heap memory? ★ MOST ASKED

Stack: one per thread; stores method calls, local variables and references; freed automatically when a method returns; `StackOverflowError` when full (deep recursion). **Heap**: shared by all threads; stores **objects**; cleaned by the **garbage collector**; `OutOfMemoryError` when full.

4 · Exception handling

11 What is the exception hierarchy?

`Throwable` → `Error` (serious JVM problems you shouldn't catch: `OutOfMemoryError`, `StackOverflowError`) and `Exception`.

`Exception` → **checked** exceptions (`IOException`, `SQLException`) and `RuntimeException` → unchecked (`NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException`).

Checked vs unchecked exceptions ★ MOST ASKED

Point	Checked	Unchecked
Checked at	Compile time — must be handled or declared	Run time — handling is optional
Parent class	Exception (not RuntimeException)	RuntimeException
Cause	External problems (file missing, network down)	Programming bugs (null access, bad index)
Examples	IOException, SQLException, ClassNotFoundException	NullPointerException, ArithmeticException

12 throw vs throws? Can we have try without catch?

throw actually throws one exception object inside a method (`throw new IllegalArgumentException()`). **throws** in the method signature declares exceptions the method may pass to its caller. A `try` needs a `catch` **or** a `finally` (or it must be try-with-resources).

13 Write a custom exception and try-with-resources. ★ MOST ASKED

A custom exception extends `Exception` (checked) or `RuntimeException` (unchecked). **try-with-resources** closes any `AutoCloseable` resource automatically.

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) { super(msg); }
}

static void withdraw(double bal, double amt) throws InsufficientBalanceException {
    if (amt > bal) throw new InsufficientBalanceException("Balance too low");
}

try (Scanner sc = new Scanner(System.in)) {   // closed automatically
    withdraw(500, 800);
} catch (InsufficientBalanceException e) {
    System.out.println(e.getMessage());       // Balance too low
}
```

5 · Collections

14 What is the Collections Framework hierarchy?

`Iterable` → `Collection` → **List** (`ArrayList`, `LinkedList`, `Vector`), **Set** (`HashSet`, `LinkedHashSet`, `TreeSet`), **Queue** (`PriorityQueue`, `ArrayDeque`).

Map is separate (not a Collection): `HashMap`, `LinkedHashMap`, `TreeMap`, `Hashtable`. `Collections` (with an s) is a utility class with `sort()`, `reverse()`, `max()`.

ArrayList vs LinkedList ★ MOST ASKED

Point	ArrayList	LinkedList
Structure	Dynamic (resizable) array	Doubly linked list
get(i)	Fast — O(1)	Slow — O(n)
Insert/delete in the middle	Slow — elements shift, O(n)	Fast once the position is found — O(1) relinking
Memory	Less (just the elements)	More (each node stores two links)
Best for	Frequent reading by index	Frequent insert/delete at the ends (also a Queue/Deque)

List vs Set vs Map · array vs ArrayList

Point	Details
List	Ordered, allows duplicates, index access
Set	No duplicates; HashSet unordered, LinkedHashSet keeps insertion order, TreeSet sorted
Map	Key–value pairs, unique keys; HashMap unordered, LinkedHashMap insertion order, TreeMap sorted by key
Array vs ArrayList	Array: fixed size, holds primitives or objects, `length`. ArrayList: grows automatically, objects only, `size()`

15 How does HashMap work internally? ★ MOST ASKED

A HashMap is an **array of buckets**. For `put(key, value)`: compute `key.hashCode()` → find the bucket index → store the entry there.

Two keys in the same bucket is a **collision** — entries form a **linked list**, which becomes a **red-black tree** when a bucket has more than 8 entries (Java 8+). `get()` finds the bucket, then uses **equals()** to find the exact key.

Default capacity 16, load factor **0.75** — when 75% full it **resizes** (doubles) and rehashes. One null key is allowed; it is not thread-safe. Always override **hashCode() and equals() together** for custom keys.

HashMap vs Hashtable vs ConcurrentHashMap

Point	HashMap	Hashtable	ConcurrentHashMap
Thread-safe	No	Yes (whole map locked)	Yes (fine-grained locking)
Null key / values	One null key, null values	None allowed	None allowed
Speed	Fast	Slow (legacy)	Fast with many threads

16 Comparable vs Comparator?

Comparable gives a class its **natural order** through `compareTo()` inside the class (String, Integer implement it). **Comparator** defines **custom orders** outside the class through `compare()` — you can have many, e.g. sort students by marks, then by name.

```
List<String> names = new ArrayList<>(List.of("Ravi", "Anu", "Bharath"));
Collections.sort(names);                              // natural: [Anu, Bharath, Ravi]
names.sort(Comparator.comparing(String::length));     // by length: [Anu, Ravi, Bharath]
```

17 Count word frequency using a HashMap.

```
Map<String, Integer> count = new HashMap<>();
for (String w : "to be or not to be".split(" ")) {
    count.put(w, count.getOrDefault(w, 0) + 1);
}
System.out.println(count.get("to") + " " + count.get("be"));   // 2 2
```

18 Fail-fast vs fail-safe iterators?

Fail-fast iterators (ArrayList, HashMap) throw `ConcurrentModificationException` if the collection is modified during iteration (except through `iterator.remove()`). **Fail-safe** iterators (ConcurrentHashMap, CopyOnWriteArrayList) work on a copy/snapshot and don't throw.

6 · Multithreading

19 How do you create a thread? Thread vs Runnable? ★ MOST ASKED

Extend `Thread` and override `run()`, or implement `Runnable` (preferred — the class can still extend another class, and the task is separate from the thread). Always start with `start()`; calling `run()` directly just runs it in the current thread.

```
Runnable task = () -> System.out.println("Running in " + Thread.currentThread().getName());
Thread t = new Thread(task, "worker-1");
t.start();      // Running in worker-1
t.join();       // wait for it to finish
```

20 What is the thread life cycle?

New (created) → **Runnable** (after `start()`, ready/running) → **Blocked / Waiting / Timed waiting** (waiting for a lock, `wait()`, `sleep()`, `join()`) → **Terminated** (run() finished).

21 What is synchronization? sleep() vs wait()?

`synchronized` lets only **one thread at a time** run a method or block on the same object's lock — preventing race conditions on shared data.

`sleep(ms)` pauses the thread **without releasing the lock** (Thread class). `wait()` releases the lock and waits until `notify()`/`notifyAll()`; it must be called inside a synchronized block (Object class).

22 What is a deadlock in Java threads?

Two threads each hold one lock and wait for the other's lock forever — e.g. T1 locks A then wants B, T2 locks B then wants A. Avoid it by acquiring locks in a **fixed order**, using timeouts (`tryLock`), and keeping synchronized blocks small.

7 · Java 8 and later

23 What are the main Java 8 features? ★ MOST ASKED

Lambda expressions, **functional interfaces**, the **Stream API**, **default and static methods in interfaces**, **method references** (`String::length`), **Optional**, and the new **Date/Time API** (`LocalDate`, `LocalDateTime`).

24 What is a lambda and a functional interface?

A **functional interface** has exactly **one abstract method** (`Runnable`, `Comparator`, `Predicate`, `Function`), optionally marked `@FunctionalInterface`. A **lambda** is a short way to implement it: `(a, b) -> a + b`.

25 What is the Stream API? Give an example. ★ MOST ASKED

A stream processes a collection in a **pipeline**: intermediate operations (`filter`, `map`, `sorted`) are lazy; a terminal operation (`collect`, `forEach`, `sum`, `count`) produces the result. Streams don't change the source collection.

```
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);
List<Integer> evenSquares = nums.stream()
        .filter(n -> n % 2 == 0)
        .map(n -> n * n)
        .collect(Collectors.toList());
System.out.println(evenSquares);   // [4, 16, 36]
```

26 What is Optional?

A container that may or may not hold a value — used to avoid `NullPointerException`. Methods: `Optional.ofNullable(x)`, `isPresent()`, `orElse(default)`, `ifPresent(...)`.

If your resume says Java full stack / Spring Boot

Expect: what Spring Boot is (auto-configuration, embedded server), `@RestController` vs `@Controller`, dependency injection with `@Autowired`, JPA/Hibernate basics, REST methods (GET/POST/PUT/DELETE) and how your API connects to the database.

Rapid-fire one-liners

Question	One-line answer
Default value of an int field / object field?	0 / null (local variables have no default).
Can we overload main()?	Yes, but the JVM only calls main(String[] args).
Size of int and long?	int 4 bytes (32-bit), long 8 bytes (64-bit).
What is a package?	A namespace grouping related classes, e.g. java.util.
What is serialization?	Converting an object to a byte stream (implements Serializable).
What is a marker interface?	An interface with no methods, e.g. Serializable, Cloneable.
Why are there no pointers in Java?	For security and simplicity — references are used instead.
What is garbage collection?	The JVM automatically frees objects that have no references.
What is an Iterator?	An object to traverse a collection: hasNext(), next(), remove().
What is the diamond operator?	<> — the compiler infers generic types: new ArrayList<>().

Common mistakes interviewers catch

- Comparing Strings with == instead of equals().
- Saying the JVM is platform independent — the bytecode is; the JVM is OS-specific.
- Calling run() and expecting a new thread — use start().
- Saying HashMap is thread-safe or keeps order.
- Mixing up final, finally and finalize().